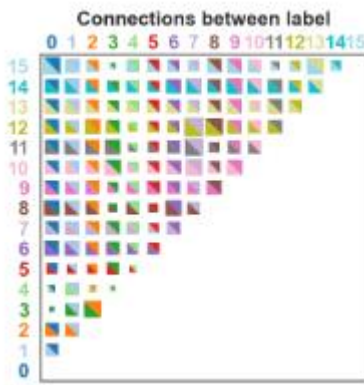


Additional Comments on BANKSY Results

1. Connections Plot Explanation

In the example Jupyter notebook, the following subfigure is presented to show the connections between BANKSY clusters:



No comments related to this are found on the paper or Jupyter notebooks. The Jupyter notebook simply calls `plot_results()` function to generate all graphs. By investigating the plotting functions (`plot_banksy.py`),

There is a function inside `plot_results` that plot the connection plot: `plot_connections`.

`Plot_connections` live in `labels.py` is listed below:

```
# Plot connectivity between labels
# -----
ax_connections = fig.add_subplot(grid[-1, -2:])

plot_connections(
    labels,
    weights_graph,
    ax_connections,
    zero_self_connections=True,
    title_str="Connections between label",
    colormap_name=c_map,
)
```

[Link](#): provides a direct access to `plot_connections` function, which includes documentation of the connections plot.

Docstring: plot the connections between labels given as a `num_label` by `num_label` matrix of connection strengths

Interpretation: Square color (split) = label color.

Square size is proportional to the connection strength.

Key takeaway:

- Visualize connection strength between labels using heatmap and shapes
- Key parameters: `zero_self_connections`: remove all self connections
- A function from BANKSY package: `interlabel_connections` is called – compute inter-label connection strength

Now, investigate the `interlabel_connections` function (in `labels.py`) which computes the connection strength between each cluster.

Key takeaway:

- Requires the weight matrix, representing connection between each cluster/label (such as the spatial graph from the BANKSY pipeline)
- converts labels into a one-hot encoded matrix and normalizes it.
- Connection matrix computation: `normalized_onehot * (weight * transpose of`

normalized_onehot)

- Computes total connection strength between each pair of labels.

The onehot matrix: **one-hot encoding**, which lets us **sum up the connections for samples belonging to the same label**.

each row is normalized by the count of that label e.g. a row [0 1 1 0 0] will be converted to [0 0.5 0.5 0 0].

Intuition behind the matrix multiplication: $O \times W \times O^T$

O: one-hot encoded label matrix, shape (# samples, # labels)

W: weight between samples (cells)

$W * O^T \rightarrow$ sum up weight from sample i to all samples that have label j (group sample connections by label)

$O \times W \times O^T \rightarrow$ sum up the contributions from all samples belonging to label i, considering connections to samples belonging to label j (aggregate label-label connection)

O acts like a filter. – final purpose: sum up sample-to-sample connections by label

2. More Examples using BANKSY, and Results Summary

Another example slideseq_v2 is presented – but again with clear boundary separation

3. Spatial weights calculations explanation

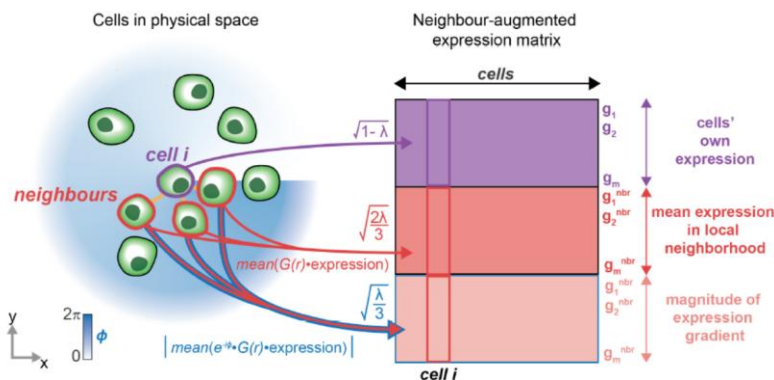
From BANKSY Jupyter notebooks: spatial weights are generated using Gaussian decay function from median distance to kNN.

Importance of Lambda (Cell-typing vs domain segmentation)

To control the relative contribution of a cell's own and neighbor features to cell-cell distances in the embedding, BANKSY uses a mixing parameter, $\lambda \in [0, 1]$, to weight the contributions of the cell-transcriptome matrix and the neighbor expression matrices (mean and AGF; Fig. 1 and Extended Data Fig. 1b). Smaller λ settings emphasize cells' own transcriptomes and thus cause cells to cluster according to cell type. In the limit, when $\lambda = 0$, BANKSY reduces to conventional nonspatial clustering. BANKSY can be switched from cell-type clustering mode to domain segmentation mode by increasing λ , which increases the influence of the neighborhood signature and causes cells to cluster according to tissue domain.

Algorithm to augment neighborhood information and calculate spatial weights:

4. Scale original expression matrix by $\sqrt{1 - \lambda}$ and neighbour expression matrix by $\sqrt{\lambda}$ and concatenate matrices to obtain **neighbour-augmented expression matrix** (n_{cells} by $2n_{genes}$) using `banksy.main.weighted_concatenate` with `neighbourhood_contribution =  $\lambda$` . These operations are performed on the numerical data `adata.X`; use `banksy.main.bansky_matrix_to_adata` to recover Anndata object with the appropriate annotations.



- Construct neighborhood graph between cells using kNN
- Compute feature-wise z-score transformation of 2 matrices: average neighborhood expression and AGF matrix
- 2 matrices scaled on mixing parameter lambda (controls relative weighting)
- Concatenate with original gene-cell expression matrix to construct 2 neighbor-augmented matrix, 1 for cell typing, 1 for domain segmentation (different lambdas)

Intuitive explanation for spatial kernels (two matrices):

Weighted Mean Kernel: This kernel computes the weighted average of gene expression in each cell's neighborhood, capturing the local transcriptomic context.

Azimuthal Gabor Filter (AGF): Inspired by plane-wave Gabor filters, the AGF captures spatial patterns and orientations in gene expression, providing a nuanced understanding of the microenvironment. (DIRECTIONAL variation)

Steps for average neighborhood expression:

Weighted neighborhood mean

For each gene q , we computed the mean expression in the neighborhood of cell $u \in \mathcal{U}$ as:

$$M_u^{(q)} = \sum_{v \in \eta_{k_{\text{geom}}}^u} g_v^{(q)} \Gamma_{uv}^{k_{\text{geom}}}$$

where $g_v^{(q)}$ is the expression of the q th gene in neighboring cell v and the term $\Gamma_{uv}^{k_{\text{geom}}}$ is a Gaussian weighting envelope defined as:

$$\Gamma_{uv}^{k_{\text{geom}}} = \frac{\exp\left(-\frac{r_{u\bar{v}}^2}{r_{u\bar{v}}^2}\right)}{\sum_{w \in \eta_{k_{\text{geom}}}^u} \left(\exp\left(-\frac{r_{uw}^2}{r_{u\bar{v}}^2}\right)\right)}$$

with $\bar{v}$ defined as the $\lfloor (m \times k_{\text{geom}})/2 \rfloor$ -th nearest neighbor (that is, for $k_{\text{geom}} = 15$, $\bar{v}$ corresponds to the 7-th nearest neighbor and $r_{u\bar{v}}$ is the distance between cells u and $\bar{v}$). This effectively makes the Gaussian modulation envelope invariant to differences in cell densities in physical space. As a default, we set $k_{\text{geom}} = 15$ in all analyses in this study. Note that the results are insensitive to moderate variations around this set point (Supplementary Figs. [27](#) and [35](#)).

Combination of Matrices: (concatenation)

The rows (features) of these two matrices, along with those of the gene–cell matrix $\mathcal{C}$ are then zero-centered and scaled to have unit variance (z-scaled). The resulting scaled $\mathcal{C}$, $\mathcal{M}$ and $\mathcal{G}$ matrices are subsequently combined to form a neighbor-augmented matrix:

$$\mathcal{B} = \begin{bmatrix} \sqrt{1 - \lambda} \mathcal{C} \\ \sqrt{\lambda/\mu} \mathcal{M} \\ \sqrt{\lambda/(2\mu)} \mathcal{G} \end{bmatrix} \in \mathbb{R}^{3p \times N} \quad (2)$$

where $\mu = 1.5$ is a normalization factor to ensure the convexity of the linear combination of distance matrices (Supplementary Section [2](#)) and $\lambda \in [0, 1] \subset \mathbb{R}$ is a mixing parameter that controls the relative weights of the three component matrices. For cell typing, we used a default value of $\lambda = 0.2$ to construct the neighbor-augmented matrix $\mathcal{B}$; for domain segmentation, we used $\lambda = 0.8$ for the creation of the corresponding $\mathcal{B}$ matrix. In either case,

Then, use this matrix for downstream PCA and Leiden.